GIT 과 PyCharm 연결

Write By. 김지훈

AIVLE

Contents

01. 버전관리 개념과 특징

02. GIT 의 기본 기능

03. 충돌과 충돌 처리

04. Branch 기능

05. 원격저장소 생성 및 연결

06. 팀원 등록과 협업

과정 OVERVIEW

GIT

- 버전관리의 개념과 특징
- GIT 의 기본기능
- 충돌과 충돌처리
- 원격저장소 생성 및 연결
- 팀원 등록과 협업

GIT 을 활용한 형상관리 기능 활용

팀원 협업을 위한 환경 구성

GIT 을 활용한 소스 형상관리 및 협업 프로젝트 실행할 수 있다.

강의 목표

GIT 을 활용한 소스 형상관리 및 협업 프로젝트 실행

형상관리와 GIT

형상관리의 개념과
형상관리의 툴을 알아보고,
GIT 과의 차이점을 확인한다.

GIT 의 기본기능과 충돌

GIT 의 기본기능과 충돌 현상
에 대해서 알아보고, 충돌 예
방법에 대해 배운다.

원격 저장소 연결과 Collaborator 등록

원격저장소를 연결하고 팀원
을 등록하여 협업을 위한 환
경을 구성한다.

01.

버전관리 개념과 특징

버전관리 시스템

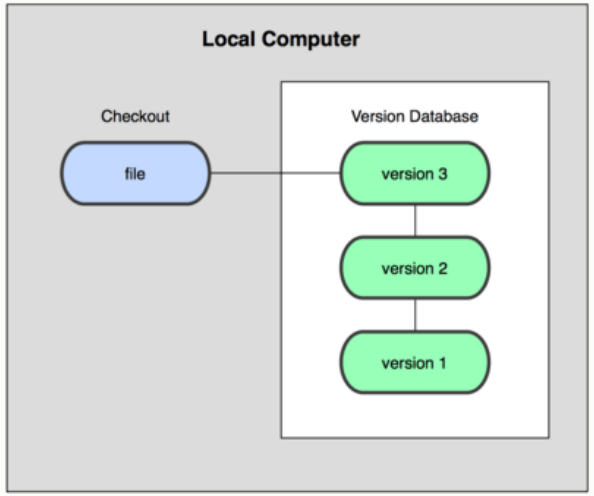
버전 관리 란?

- 파일변화를 시간에 따라 기록하여 특정 시점의 버전을 다시 불러 올 수 있는 시스템
- 버전 관리에는 로컬, 중앙집중식, 분산버전 관리 시스템 등이 있음

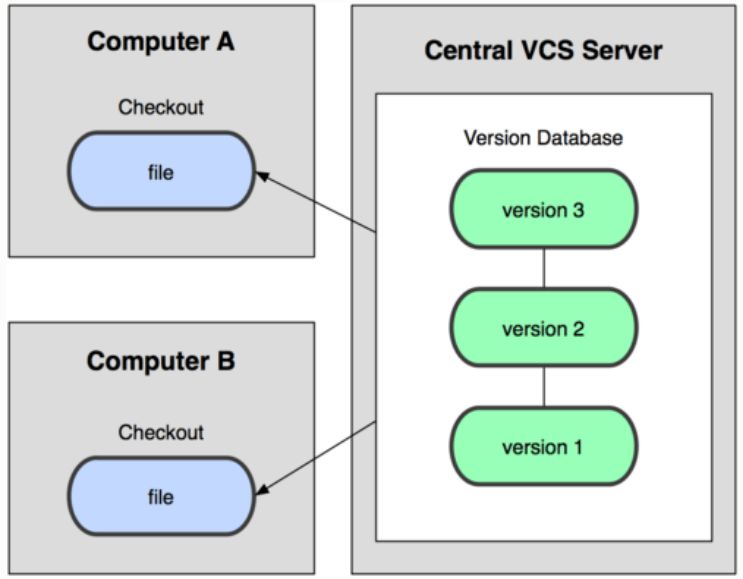
방식	특징	대표 TOOL
로컬 버전 관리	파일의 변경 사항을 DB 에 기록	NCS
중앙 집중 버전 관리	각 클라이언트의 변경 내용을 중앙 서버에서 관리	CVS, SVN
분산 버전 관리	각 클라이언트 들이 중앙서버의 저장소를 통 째로 복사	Git, Mecurial, Bazaar

버전관리 시스템

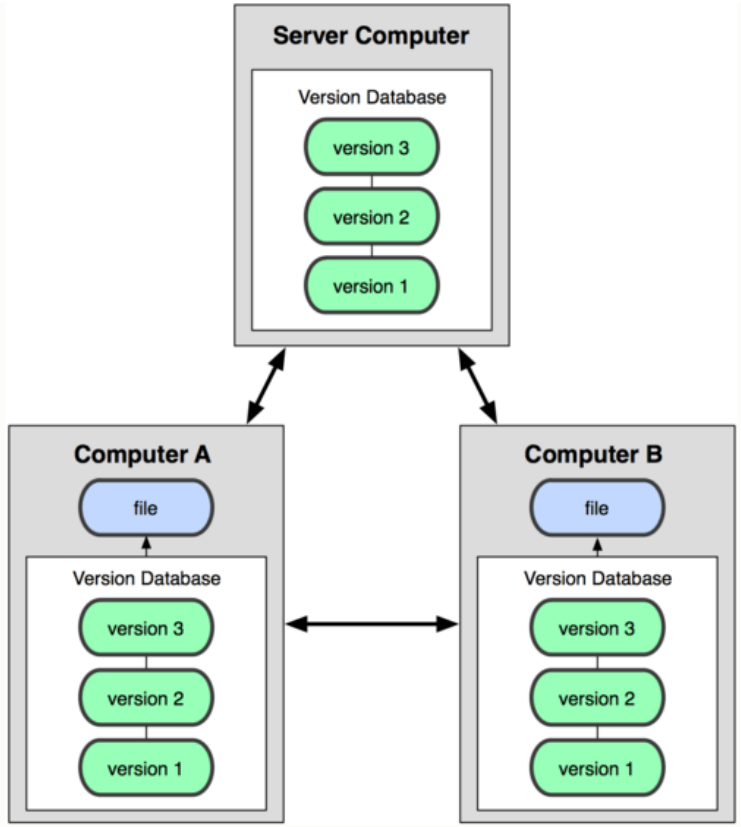
버전 관리 란?



로컬



중앙 집중



분산

버전관리 시스템

GIT 의 기초

- Git 은 현재 파일이 존재하는 그 순간을 중심으로 함
- Git은 로컬을 베이스로 하므로 **인터넷 연결 없이 버전관리가 가능**
- Git은 파일을 **Committed, Modified, Staged** 세 가지 상태로 관리
- push 를 통해 원격 저장소에 공유가 가능

상태	설명	저장위치
1. Un staged	무언가 작성하고 막 저장한 상태	로컬
2. Staged	현재 수정한 파일을 곧 commit 할 상태	로컬
3. Committed	변경내용이 로컬에 저장된 상태	로컬
4. Push	변경내용이 원격 저장소에 업로드	원격

버전관리 시스템

GIT 의 설치 여부

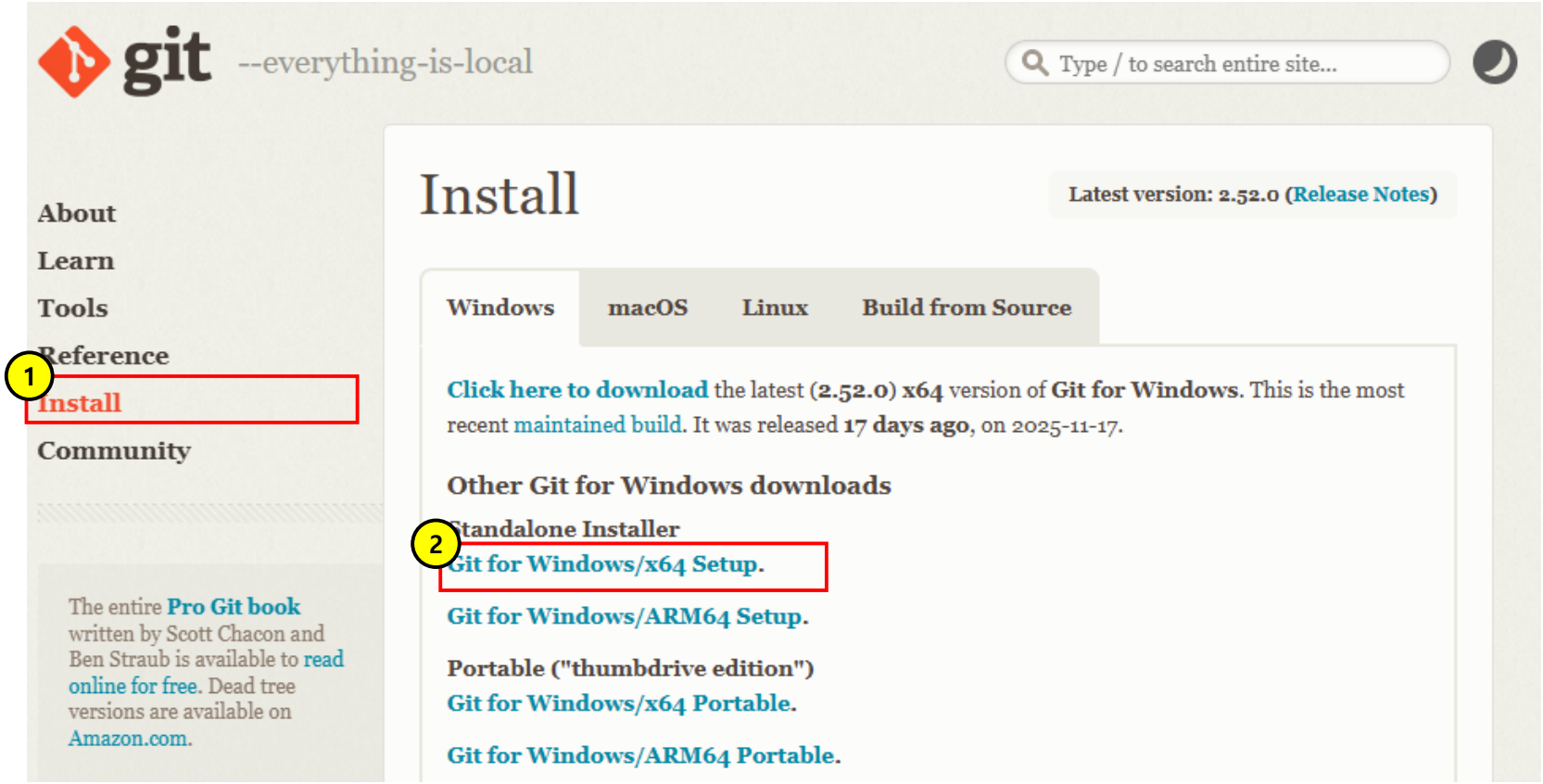
- Git 의 설치 여부는 명령어를 통해 알 수 있음

```
C:₩Users₩M>git --version
git version 2.45.1.windows.1
```

버전관리 시스템

GIT 설치 방법 1

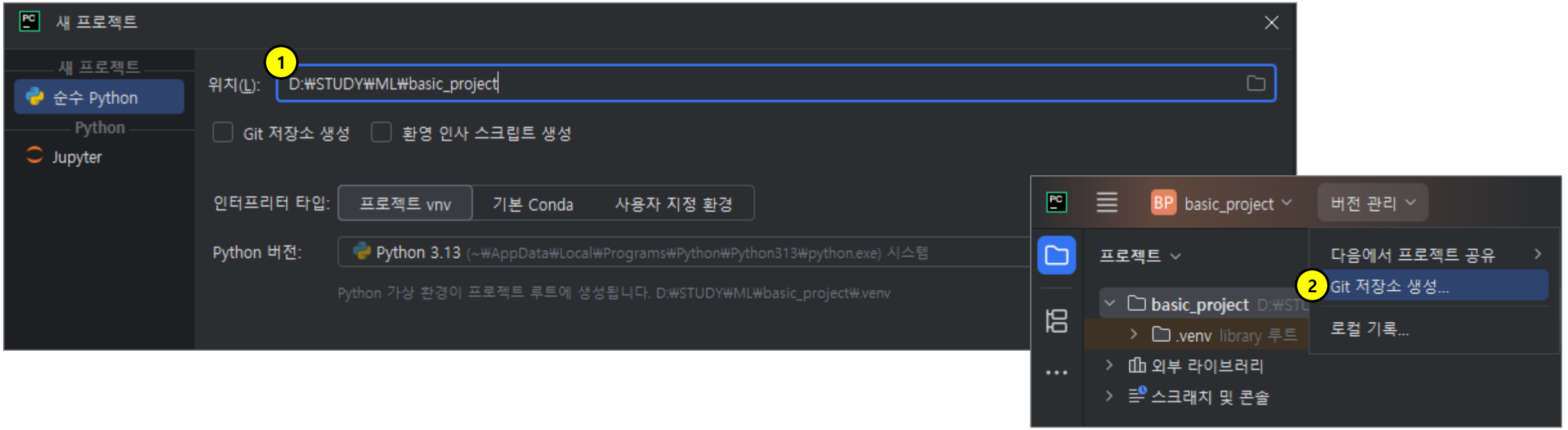
- Git 사이트(https://git-scm.com/install/windows)에서 해당하는 OS 버전을 선택 후 다운로드 이후 설치



버전관리 시스템

GIT 설치 방법 2

- Py cham 실행 후 신규 프로젝트 생성
- 생성 후 버전관리 > Git 저장소 생성 후 해당 프로젝트 선택

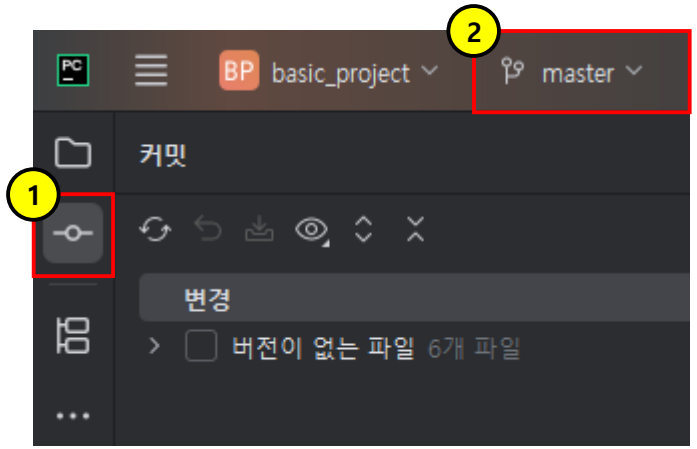


버전관리 시스템

GIT 설치 방법 2

- Git 이 없어 실패할 경우(설치가 안되어 있어서) 다운로드 및 설치 실행
- 다시 git 으로 관리할 프로젝트 지정
- Git 버튼(1번) 을 누르면 commit 을 위한 창이 나타나고 현재 branch(2번) 도 보인다면 성공적으로 설치

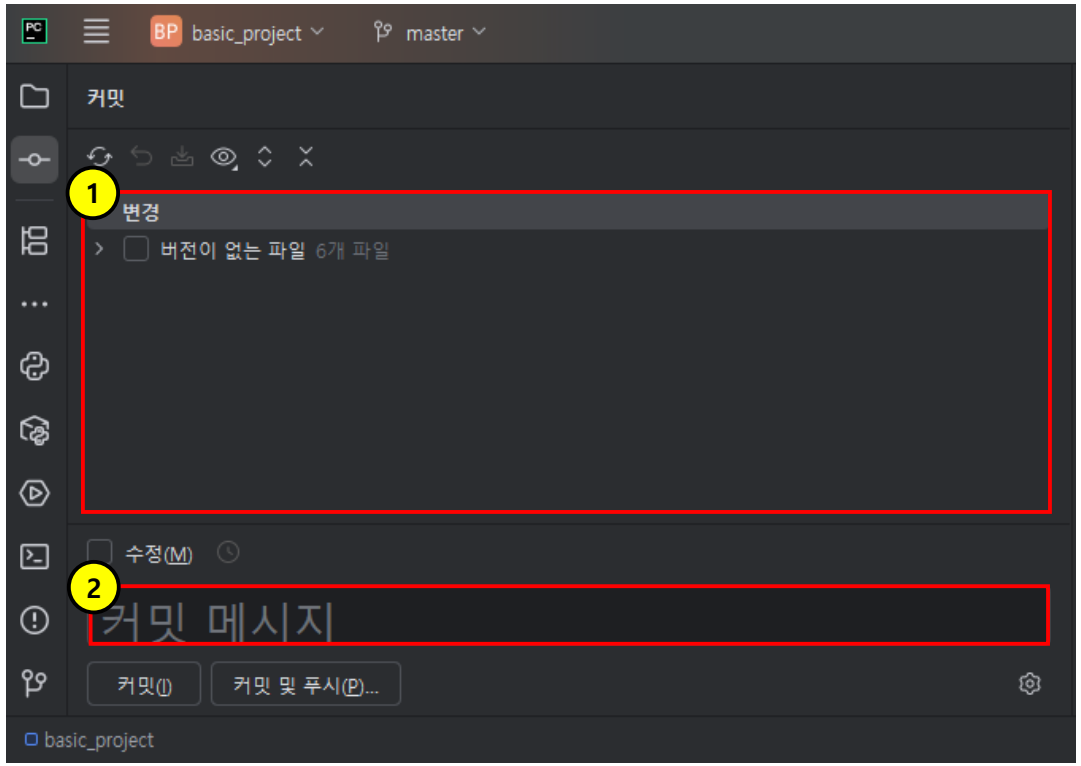




버전관리 시스템

GIT 설치 방법 2

- (1번) 변경된 파일 목록을 보여줌
- (2번) commit 시 남길 메시지를 적는 란



Summary

버전관리 개념

- 파일변화를 시간에 따라 기록하여 특정 시점의 버전을 다시 불러 올 수 있는 시스템

방식	특징
로컬 버전 관리	파일의 변경 사항을 DB 에 기록
중앙 집중 버전 관리	각 클라이언트의 변경 내용을 중앙 서버에서 관리
분산 버전 관리	각 클라이언트 들이 중앙서버의 저장소를 통 째로 복사

GIT

- 로컬을 베이스로 하므로 인터넷 연결 없이 버전관리가 가능
- 파일을 Committed, Modified, Staged 세 가지 상태로 관리
- push 를 통해 원격 저장소에 공유가 가능

상태	설명
Un staged	무언가 작성하고 막 저장한 상태
Staged	현재 수정한 파일을 곧 commit 할 상태
Committed	변경내용이 로컬에 저장된 상태
Push	변경사항 history 를 원격 서버에 업로드

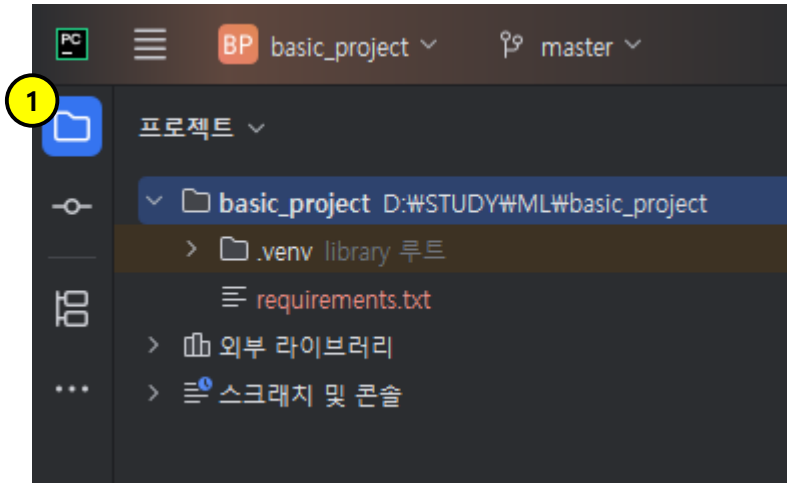
02.

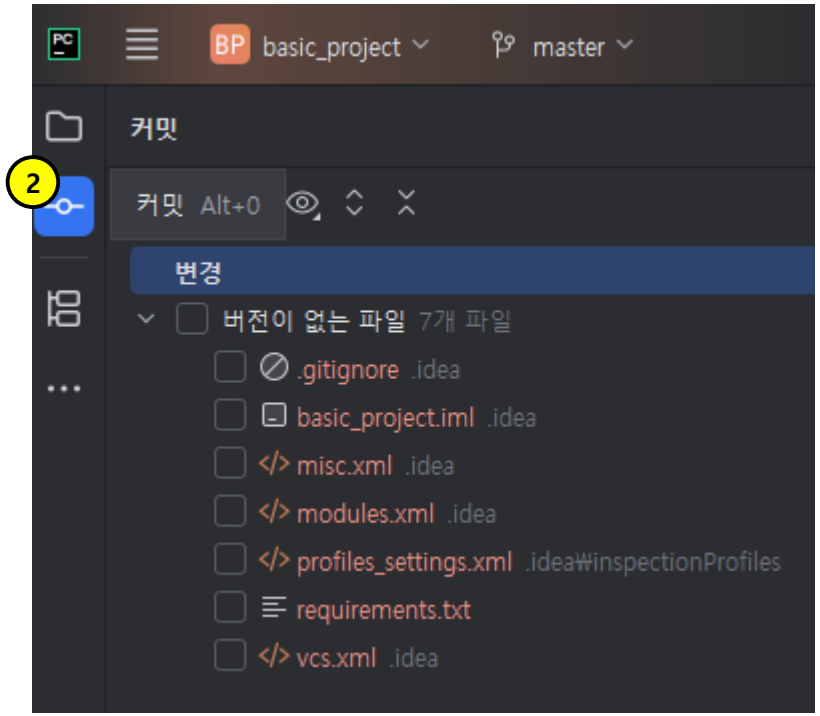
GIT 의 기본 기능

GIT 의 기본 기능

신규 파일 인식

- requirements.txt 파일을 인식 시켜 보자
- (1번)프로젝트와 (2번)커밋 창에 따라 다른 내용이 보여 짐
- **새로 인식된 파일은 빨강색**으로 표기됨



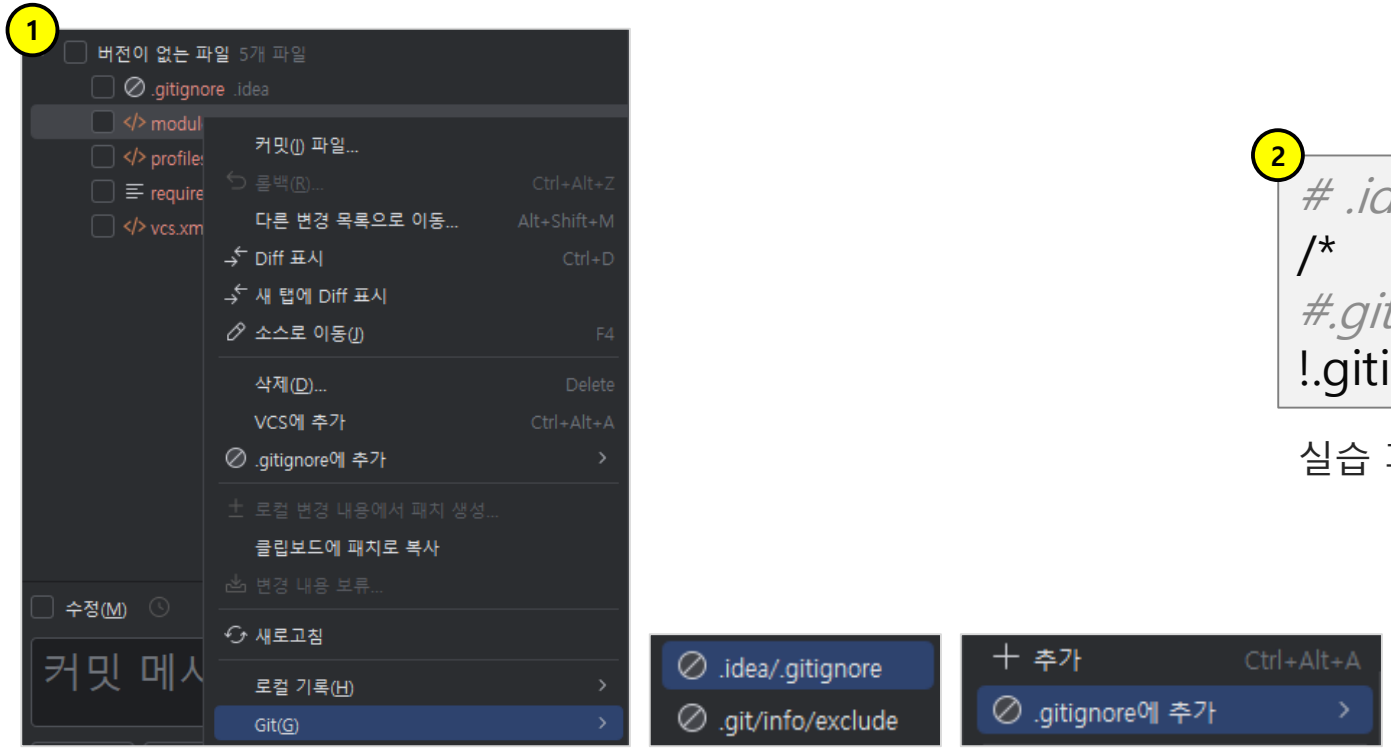


실습 파일 : 01_GIT/01_start/requirements.txt

GIT 의 기본 기능

무시(ignore)

- 변화를 추적할 필요가 없거나 공유하고 싶지 않은 파일의 경우 무시(ignore) 가 가능
- (1번) 원하는 파일 우 클릭 > Git > .gitignore 에 추가 > .idea/.gitignore
- (2번) .gitignore 파일에 무시하고 싶은 파일 명시 가능



2

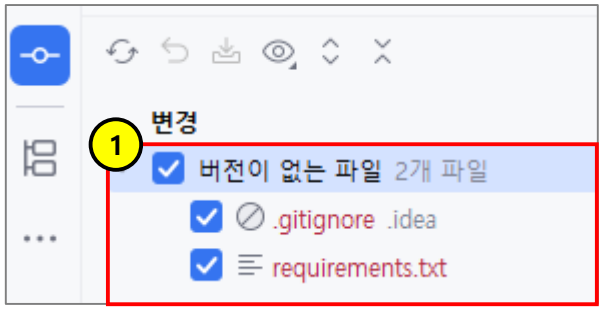
```
# .idea 폴너 내부의 모든파일 무시
/*
#.gitignore 파일은 제외
!.gitignore
```

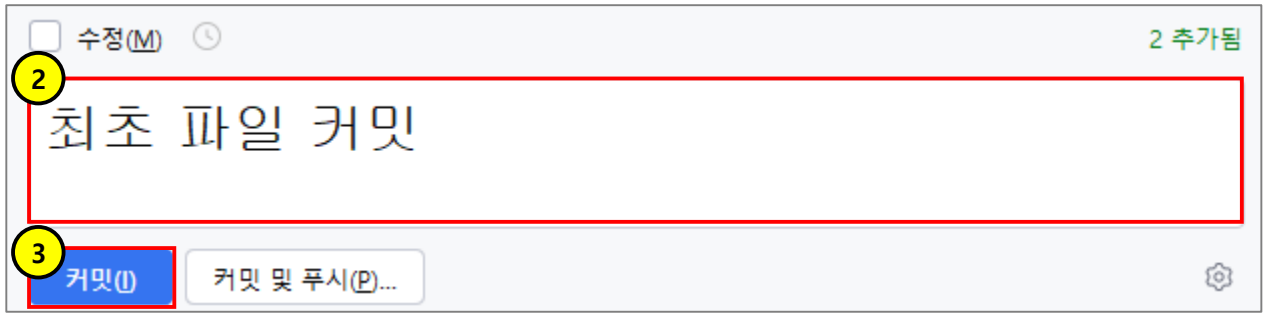
실습 파일 : 01_GIT/01_start/.idea/.ignore

GIT 의 기본 기능

commit

- 생성된 .ignore 파일과 requirements.txt 파일을 commit 해 보자





- 그리고 requirements.txt 파일의 의존성을 설치해 주자

```
pip install -r requirements.txt
```

GIT 의 기본 기능

commit

- (1번)main.py 파일 생성 후 아래 내용을 작성 후 commit
- (2번)view 폴더 생성 후 index.html 작성 후 commit

1

```python
from fastapi import FastAPI
from starlette.responses import RedirectResponse
from starlette.staticfiles import StaticFiles

app = FastAPI()

app.mount("/view", StaticFiles(directory="view"))

@app.get("/")
def root():
    return RedirectResponse("/view/index.html")
```

실습 파일 : 01_GIT/01_start/main.py

2

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>

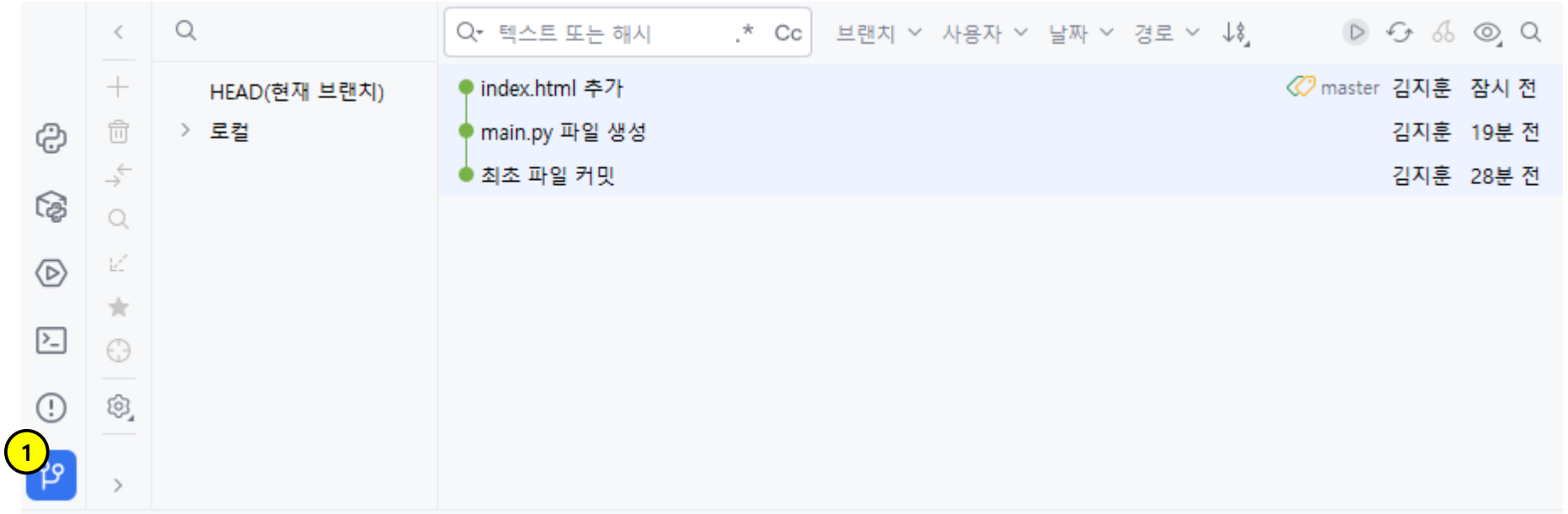
</body>
</html>
```

실습 파일 : 01_GIT/01_start/view/index.html

GIT 의 기본 기능

commit

- 하단의 버튼을 누르면 commit 을 수행한 branch history 확인 가능
- 각 점(commit point) 를 연결한 라인을 branch 라고 부름

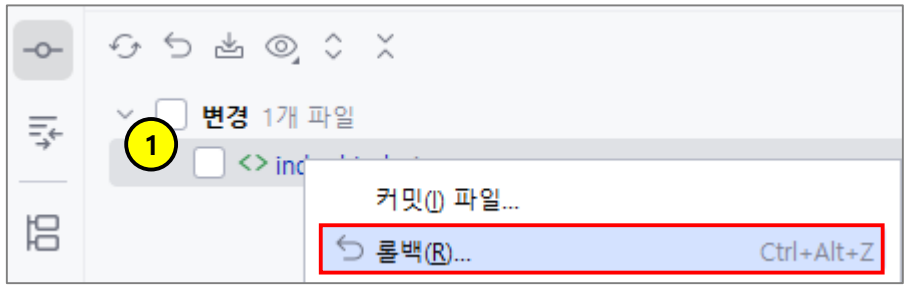


GIT 의 기본 기능

Commit 전 되돌리기(roll back)

- Commit 전 내용을 롤백(roll back) 을 통해 되돌릴 수 있음
- 변경된 파일 우 클릭 > 롤백

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <h1>어차피 취소될 내용
</h1>
</body>
</html>
```



실습 파일 : 01_GIT/01_start/view/index.html

GIT 의 기본 기능

Commit 한 내용 되돌리기(branch initialize)

- 우선 commit 을 아래와 같이 진행 해 보자

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
    <h3>commit 한 내용을 취소하고 싶다면</h3>
    <p>1. 브랜치 초기화</p>
    <p>2. reverse commit</p>
</body>
</html>
```

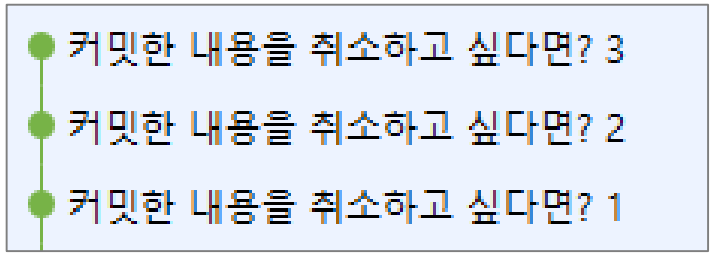
1

2

```
<h3>commit 한 내용을 취소하고 싶다면</h3>
<p>1. 브랜치 초기화 - 특정 지점으로 초기화 하는 내용</p>
<p>2. reverse commit</p>
```

3

```
<h3>commit 한 내용을 취소하고 싶다면</h3>
<p>1. 브랜치 초기화 - 특정 지점으로 초기화 하는 내용</p>
<p>2. reverse commit - 이건 아직 잘 모르겠네</p>
```

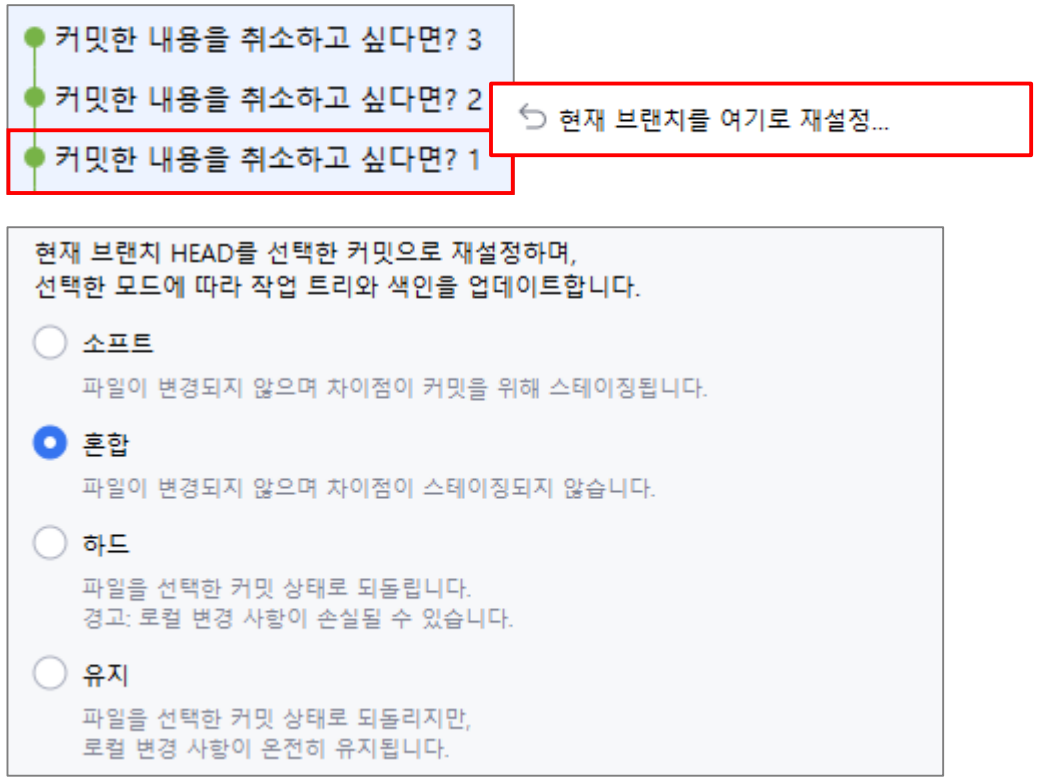


실습 파일 : 01_GIT/01_start/view/index.html

GIT 의 기본 기능

Commit 한 내용 되돌리기(branch initialize)

- Git history 창에서 우 클릭 > 현재 브랜치를 여기로 재설정 선택

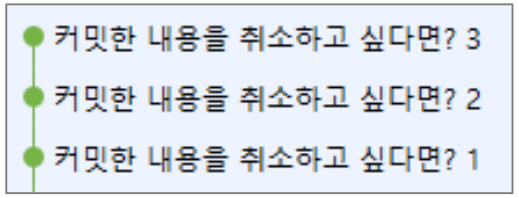


상태	설명
소프트	소스 유지, 작업을 Staging 상태로 둠
혼합	소스 유지, 작업을 Un Staging 상태로 둠
하드	작업을 폐기
유지	혼합과 유사

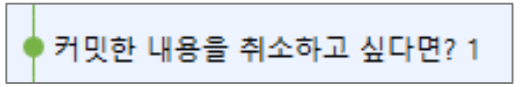
GIT 의 기본 기능

Commit 한 내용 되돌리기(branch initialize)

- 수행 결과를 살펴 보자




```
<body>
    <h3>commit 한 내용을 취소하고 싶다면</h3>
    <p>1. 브랜치 초기화 - 특정 지점으로 초기화 하는 내용</p>
    <p>2. reverse commit - 이건 아직 잘 모르겠네</p>
</body>
```




```
<body>
    <h3>commit 한 내용을 취소하고 싶다면</h3>
    <p>1. 브랜치 초기화</p>
    <p>2. reverse commit</p>
</body>
```

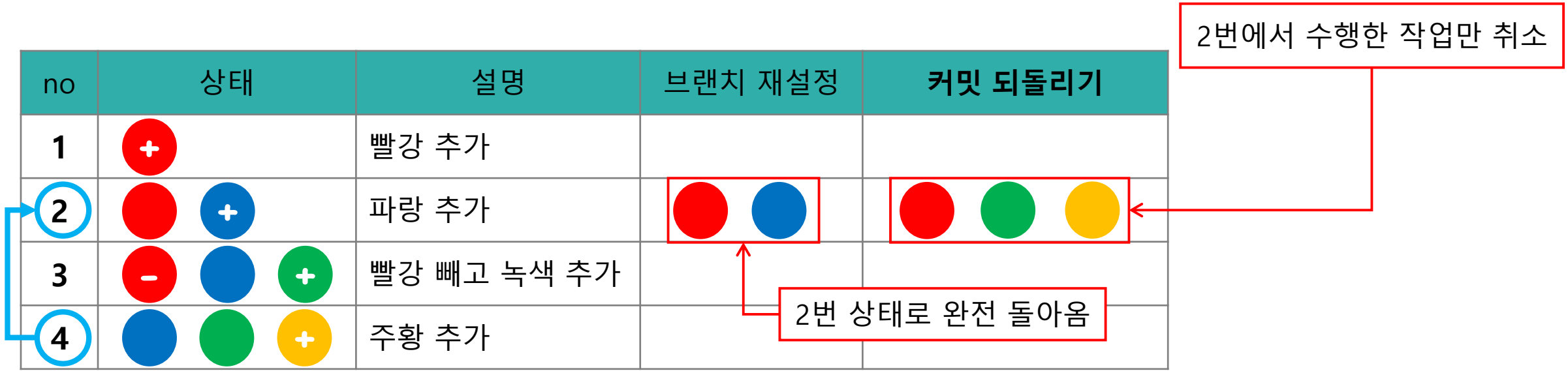
실습 파일 : 01_GIT/01_start/view/index.html

GIT 의 기본 기능

Commit 한 내용 되돌리기(reverse commit)

- Commit 된 내용을 되돌리는 방법은 branch initialize 외에도 reverse commit 이 존재함
- 둘의 차이를 살펴 보자

no	상태	설명	브랜치 재설정	**커밋 되돌리기**
1		빨강 추가		
2		파랑 추가		
3		빨강 빼고 녹색 추가		
4		주황 추가		



GIT 의 기본 기능

Commit 한 내용 되돌리기(reverse commit)

- Commit 되돌리기를 위해 다음 내용을 순차적으로 commit 해 보자

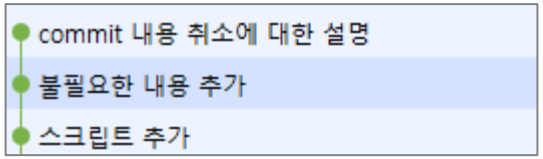
1
```
<script>
    let msg = "simple message";

    if(msg != ""){
        alert(msg);
    }
</script>
```

2
```
<h3>
      commit 한 내용을 취소하고 싶다면
  </h3>
    <p>1. 브랜치 초기화</p>
    <p>2. reverse commit</p>
    <p>불필요한 내용 추가</p>
```

3
```
<h3>commit 한 내용을 취소하고 싶다면</h3>
<p>1. 브랜치 초기화 - 특정 지점으로 초기화 하는 내용</p>
<p>2. reverse commit - 특정 커밋 작업 내용을 취소</p>
<p>불필요한 내용 추가</p>
```

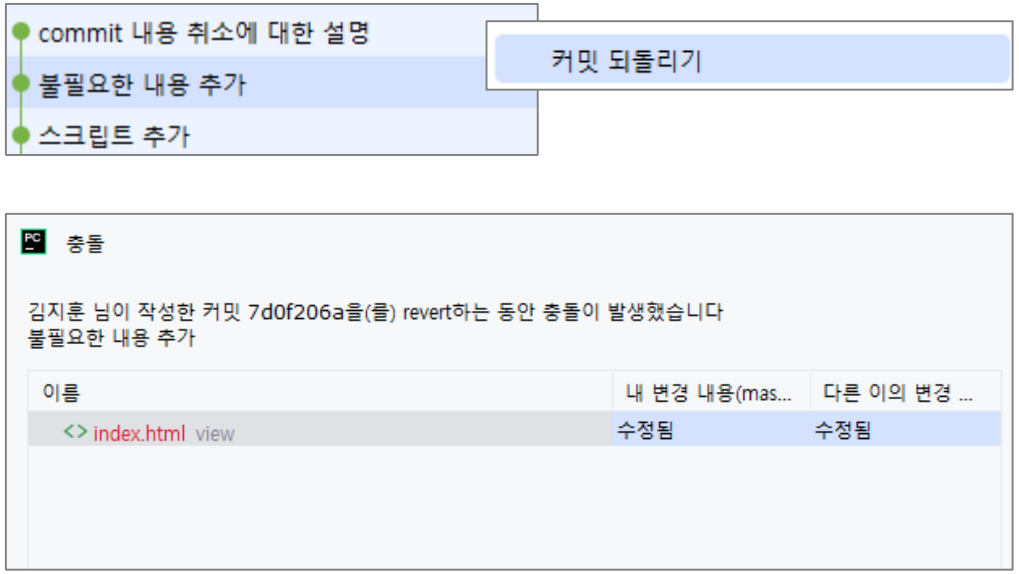
실습 파일 : 01_GIT/01_start/view/index.html



GIT 의 기본 기능

Commit 한 내용 되돌리기(reverse commit)

- 불필요한 내용 추가한 부분만 취소하기 위해 커밋 되돌리기를 실행해보자
- 충돌(complicit) 이 발생하는데, 이는 서로 다른 유저가 같은 곳을 수정할 때 주로 발생
- 아래는 이전버전과 되돌리려는 버전이 같은 지점이어서 발생한 문제




```
<body>
    <h3>commit 한 내용을 취소하고 싶다면</h3>
<<<<<<< HEAD
    <p>1. 브랜치 초기화 - 특정 지점으로 초기화 하는 내용</p>
    <p>2. reverse commit - 특정 커밋 작업 내용을 취소</p>
    <p>불필요한 내용 추가</p>
=======
    <p>1. 브랜치 초기화</p>
    <p>2. reverse commit</p>
>>>>>>> parent of 7d0f206 (불필요한 내용 추가)
</body>
```

GIT 의 기본 기능

충돌해결

- 첫번째 해결 방법은 두 부분 중 하나를 선택한 다음 나머지를 지워주는 방법

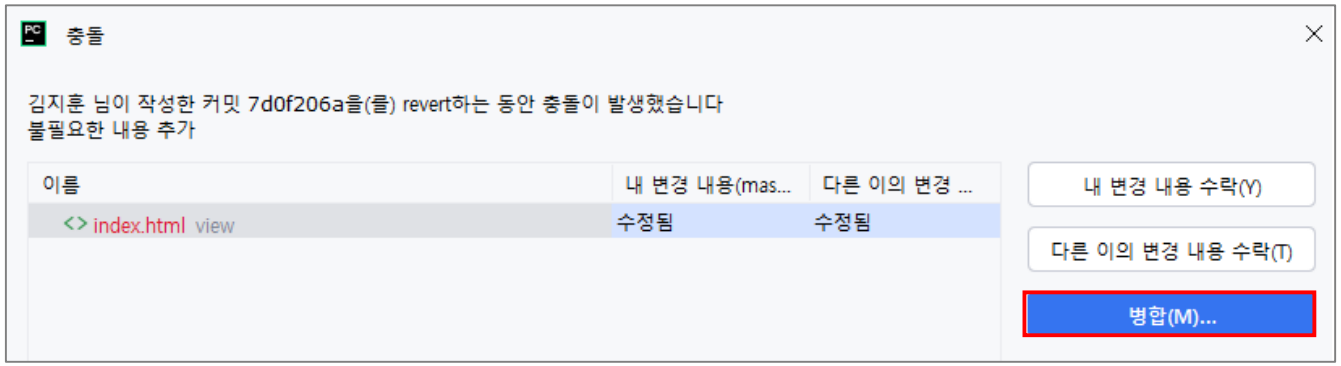
```
<body>
    <h3>commit 한 내용을 취소하고 싶다면</h3>
<<<<<<< HEAD
    <p>1. 브랜치 초기화 - 특정 지점으로 초기화 하는 내용</p>
    <p>2. reverse commit - 특정 커밋 작업 내용을 취소</p>
    <p>불필요한 내용 추가</p>
=======
    <p>1. 브랜치 초기화</p>
    <p>2. reverse commit</p>
>>>>>>> parent of 7d0f206 (불필요한 내용 추가)
</body>
```

```
<body>
    <h3>commit 한 내용을 취소하고 싶다면</h3>
    <p>1. 브랜치 초기화</p>
    <p>2. reverse commit</p>
</body>
```

GIT 의 기본 기능

충돌해결

- 두번째 해결 방법은 병합을 선택 한 뒤 원하는 부분을 선택 후 적용을 통해 해결





Summary

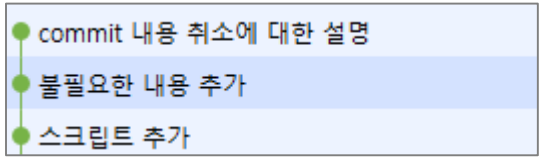
ignore

- Git 에서 파일 변화를 감지 하지 않도록 무시할 수 있는 기능
- .gitignore 파일을 통해 패턴 형태로 등록 및 삭제 가능

```
# .idea 폴너 내부의 모든파일 무시
/*
#.gitignore 파일은 제외
!.gitignore
```

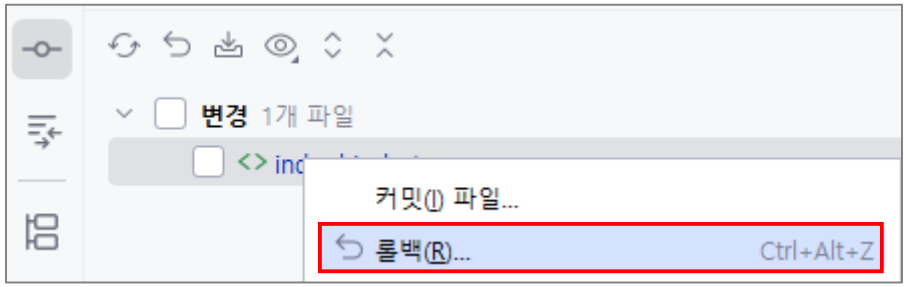
commit

- Commit 은 통해 변경 내용을 확정하고 history 를 남기는 기능
- Commit 전/후 에 따라 각각의 변경내용 취소 기능이 있음



rollback

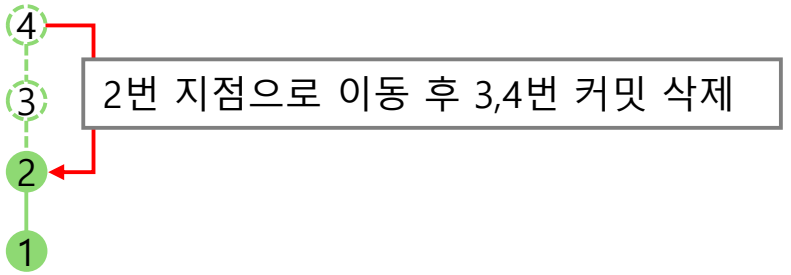
- Rollback 은 commit 전 변경 사항을 되돌리는 기능



Summary

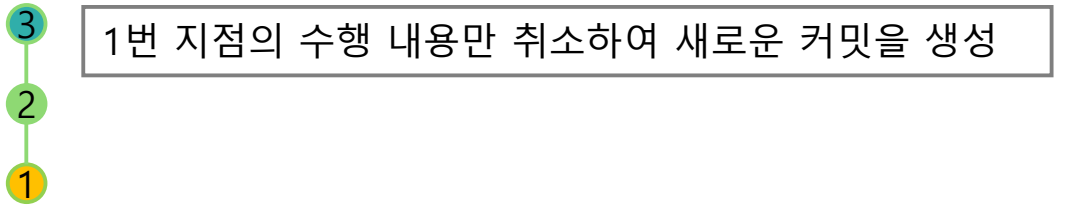
브랜치 재설정

- 특정 커밋 한 부분으로 되돌아가 이전 commit 을 모두 지움
- 해당 커밋 상태로 완전히 초기화 되는 개념



커밋 내용 되돌리기

- 특정 커밋 에서 수행한 내용만 취소



충돌 및 처리

- 충돌은 같은 지점에서 여러 개의 변경점이 나타날 때 발생
- 주로 한 부분에 여러 유저가 수정을 할 때 발생
- 충돌한 두둔중 하나를 선택해서 해결 가능

```
<body>
  <h3>commit 한 내용을 취소하고 싶다면</h3>
<<<<<<< HEAD
  <p>충돌 지점 1</p>
=======
  <p>충돌 지점 2</p>
>>>>>>> parent of 7d0f206 (불필요한 내용 추가)
</body>
```

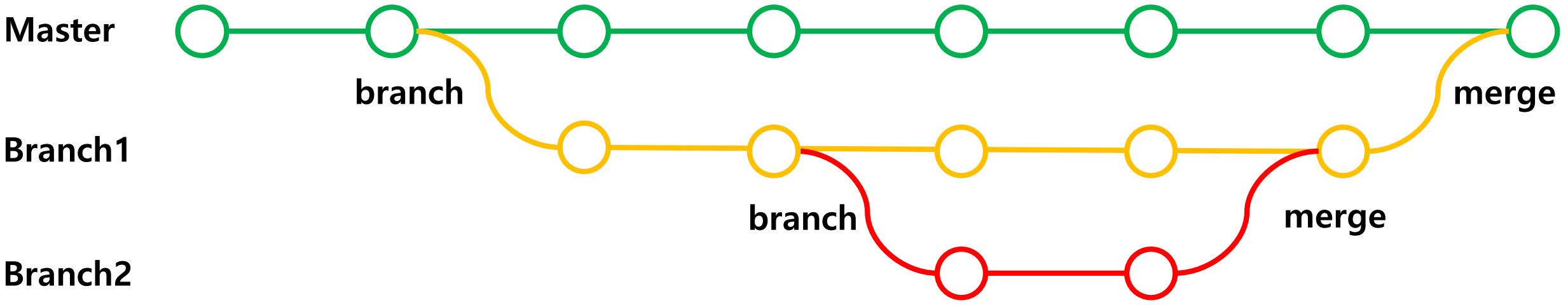
03.

branch

branch

Branch 란?

- Branch 란 서로 다른 컨셉의 소스를 관리할 때 주로 사용
- Online 에서 여러 팀원이 작업을 하면 자연스럽게 팀원 별로 branch 가 생성됨



branch

Branch 생성

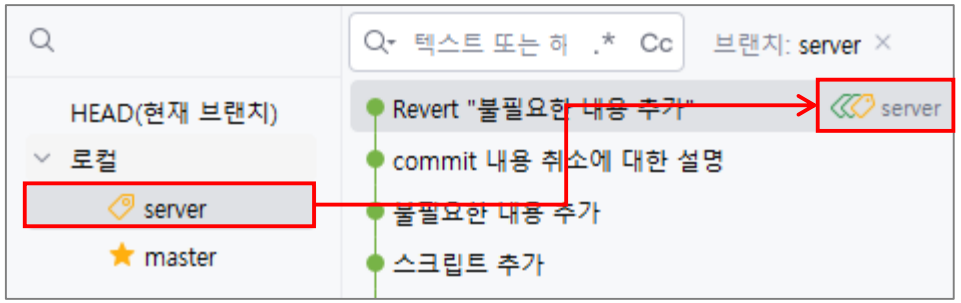
- 새로운 branch 를 생성해 보자
- (1번) branch 를 나눌 commit 지점에서 우 클릭 후 새 브랜치 선택
- (2번) branch 이름 설정 후 생성



branch

Branch 생성

- 원하는 브랜치를 더블 클릭하면 브랜치를 변경해 보자
- Server branch 에서 main.py 를 commit 해 보자






```python
@app.get("/one")
def step_one():
    return {"step":"one"}

@app.get("/two")
def step_two():
    return {"step":"two"}

@app.get("/three")
def step_three():
    return {"step":"three"}
```

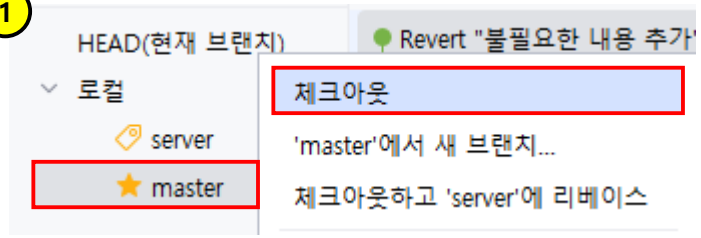
실습 파일 : 01_GIT/01_start/main.py

branch

Branch 생성

- master branch 에서 client.html 을 commit 해 보자
- (1번) master 우 클릭 > 체크아웃 > client.html 수정 및 commit

1



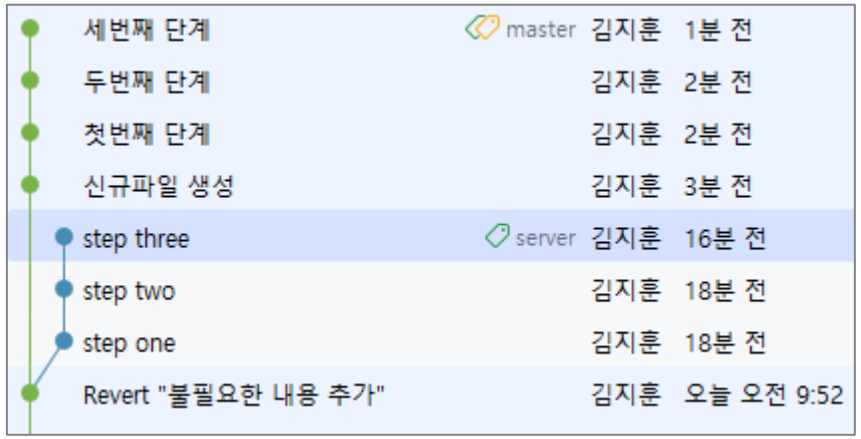
2

```
<a href="/one">첫번째  단계</a>

<a href="/two">두번째  단계</a>

<a href="/three">세번째  단계</a>
```

실습 파일 : 01_GIT/01_start/view/client.html



branch

Branch 병합

- 현재 master 와 server 를 각각 체크 아웃하면 client.html 유무와 main.py 내용 등이 다르다 .
- 이 두 내용을 병합 해 보자(master 를 기준으로 server 를 병합)
- (1번) master 브랜치 체크 아웃
- (2번) 병합할 브랜치 우클릭 후 병합 메뉴 선택

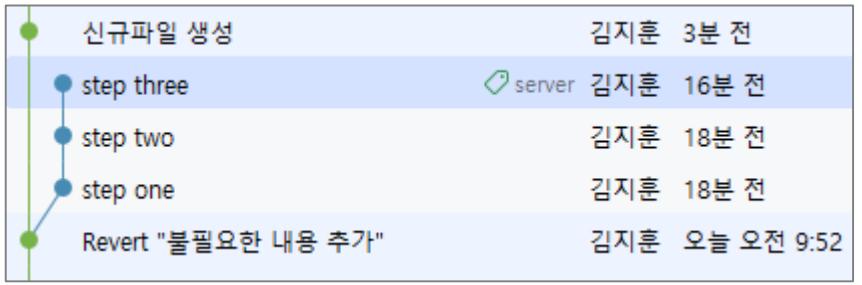




Summary

branch

- 서로 다른 버전 등의 개발을 진행 시 branch 로 구분하여 관리 가능
- 체크아웃을 통해 특정 브랜치 상태의 소스로 이동 가능



merge

- 분리된 branch 를 병합 기능을 통해 합칠 수 있음
- 이때 기준이 될 branch 를 체크아웃 이후 진행



03.

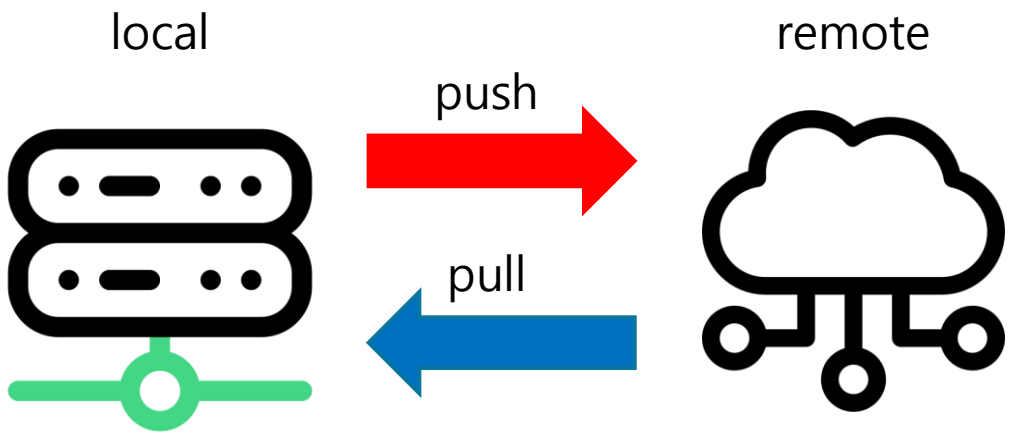
원격 저장소와 협업

원격 저장소

Git 의 저장 전략

- GIT 은 commit 을 통해 로컬 저장소에 변경 사항을 저장함
- Push 와 pull 을 통해 원격 저장소로의 저장과 호출을 할 수 있음

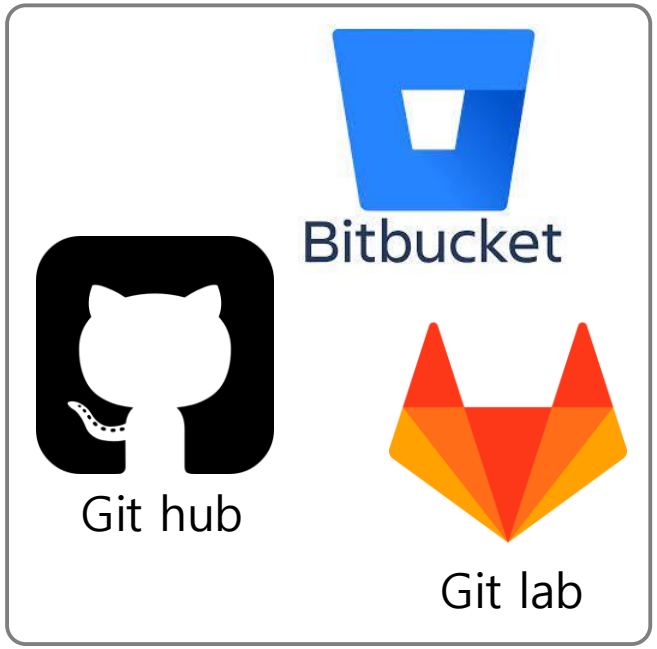
상태	설명	저장위치
Commit	변경내용이 로컬에 저장된 상태	로컬
Push	변경내용이 원격 저장소에 업로드	원격
Pull	원격 저장소에 저장된 변경 내용 다운로드	원격

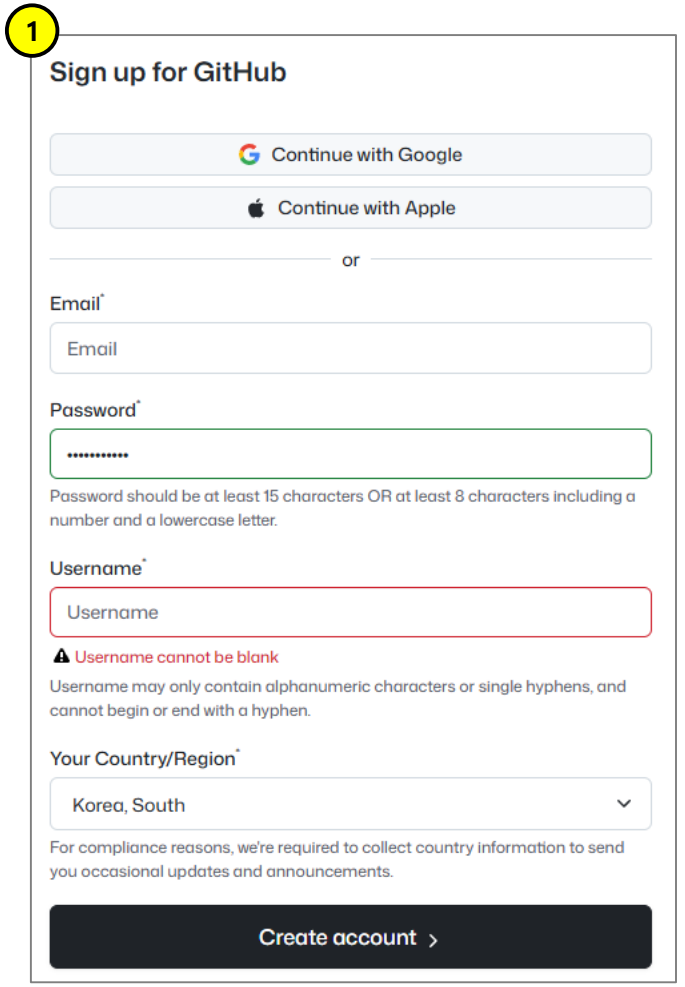


원격 저장소

Git Hub 원격 저장소 등록

- 원격 저장소는 로컬에 있는 내용을 공유 할 수 있도록 공통으로 저장해 주는 곳
- Git hub, Bit Bucket, Git lab 등 다양한 서비스가 존재함
- 이중에서 가장 대중화 된 git hub 를 사용할 예정
- (1번) git hub 를 먼저 회원가입 후 로그인 해 주자

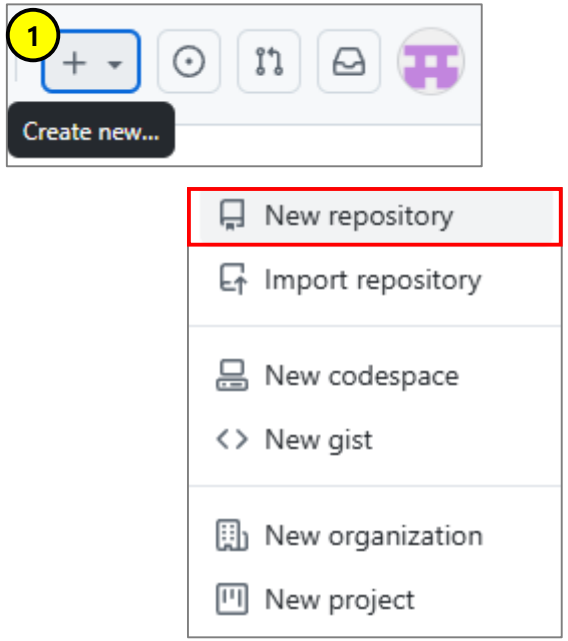


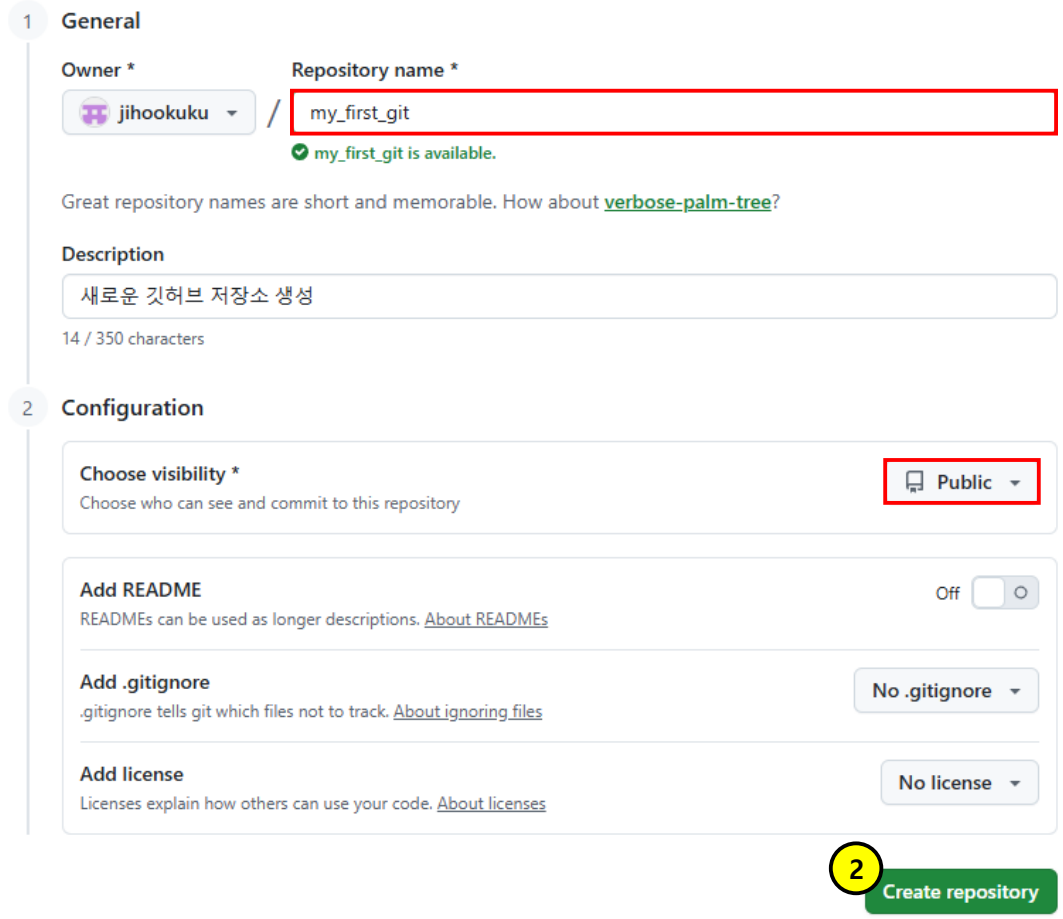


원격 저장소

Git Hub 원격 저장소 등록

- (1번) 우측 상단에 + 아이콘을 눌러 new repository 선택
- (2번) 저장소 이름과 공개여부 설정 후 create repository 클릭

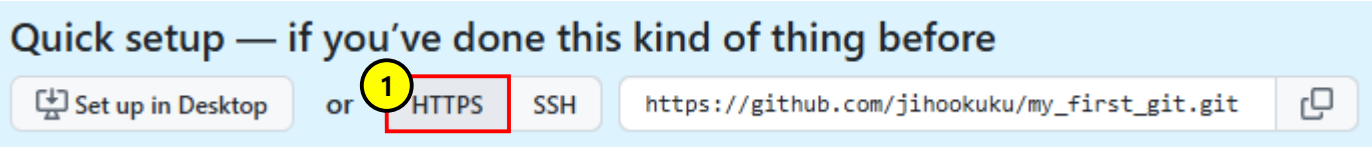




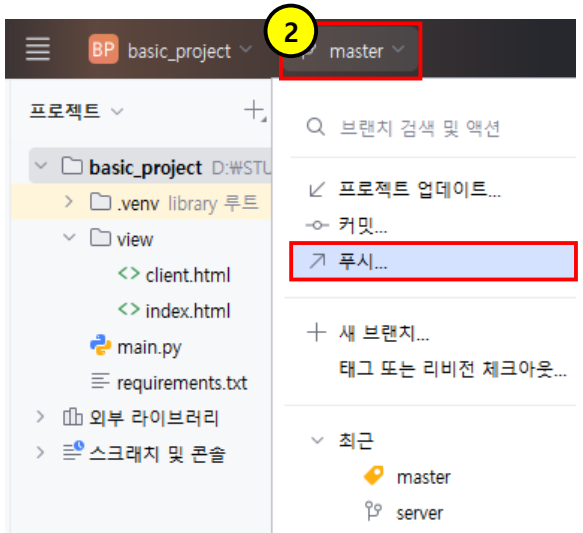
원격 저장소

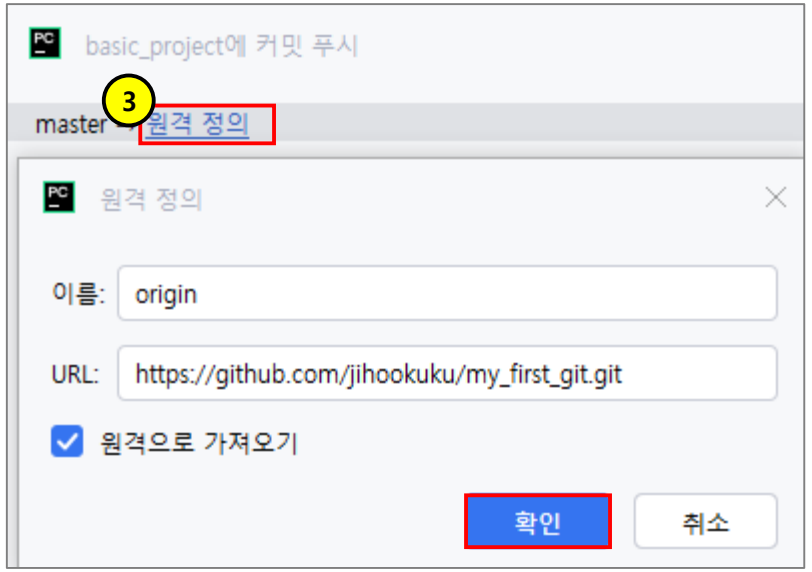
Git Hub 원격 저장소 등록

- (1번) 생성 후 https 의 git 주소를 복사



- (2번) branch 선택 > 푸시 선택
- (3번) 원격 저장소에 대한 정보 추가

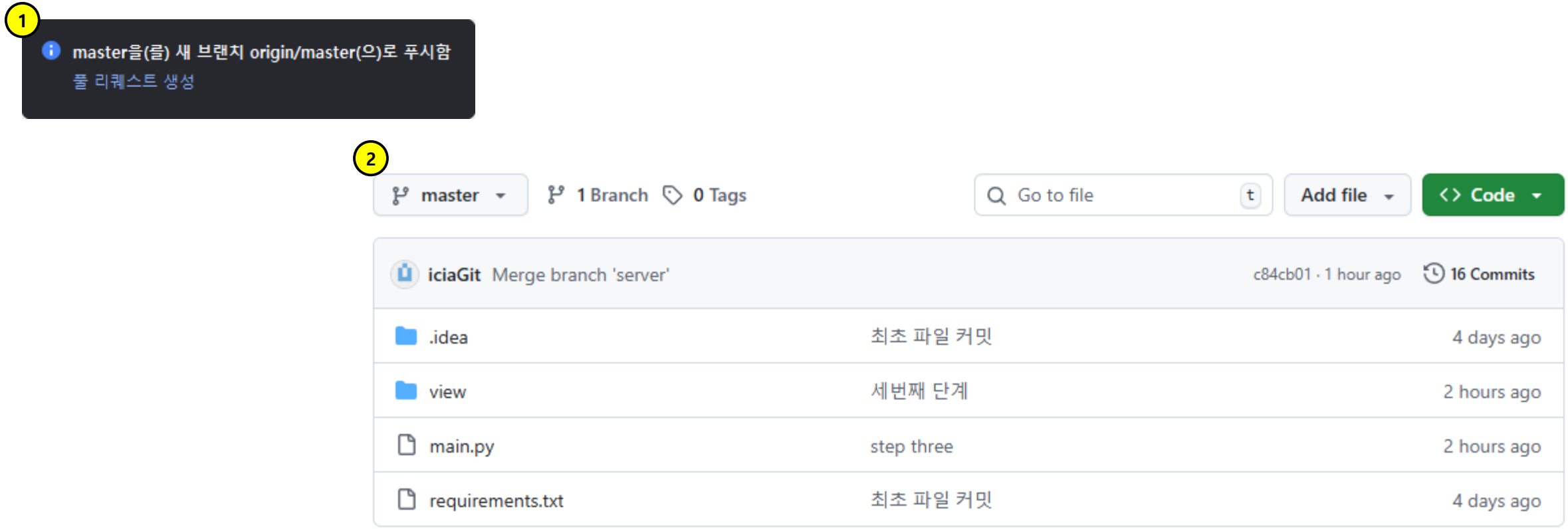




원격 저장소

Git Hub 원격 저장소 등록

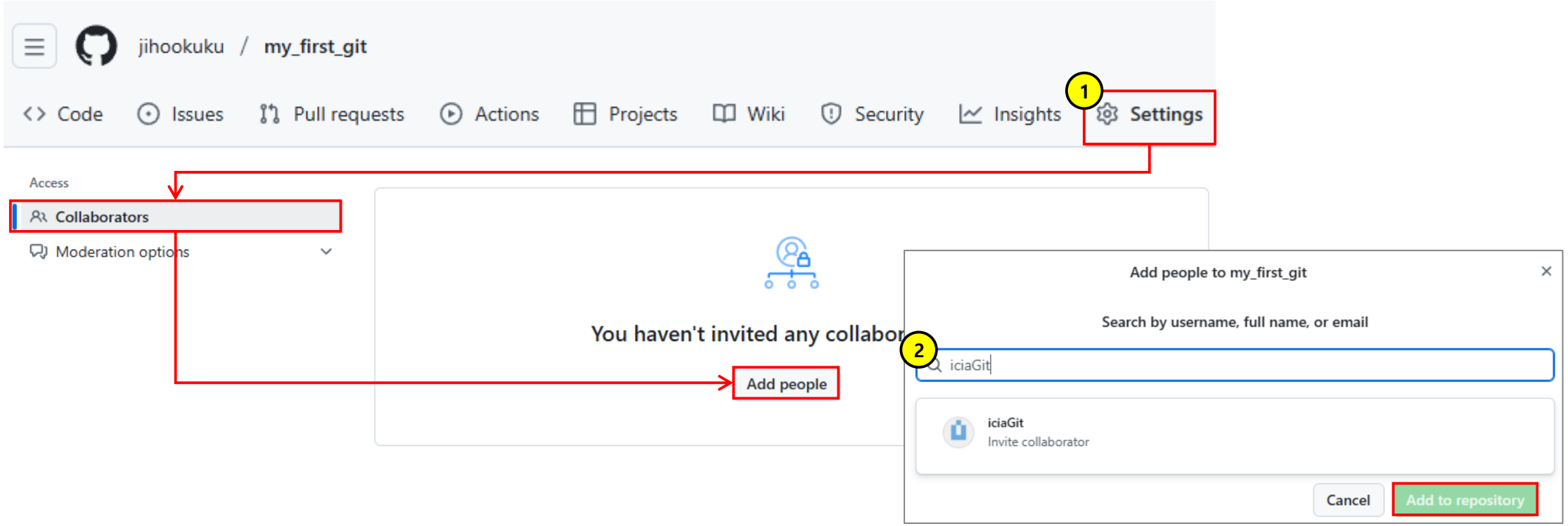
- (1번) push 가 완료 되면 안내가 우측 하단에 나타남
- (2번) 생성한 git 주소에 가보면 푸시된 히스토리가 업로드



협업

팀원들과 저장소 공유

- 팀원들과 원격 저장소를 공유하기 위해서는 collaborator 등록이 필요
- (1번) Settings > Collaborators > Add people
- (2번) 팀원들의 git hub 아이디를 받아 입력 후 Add to repository



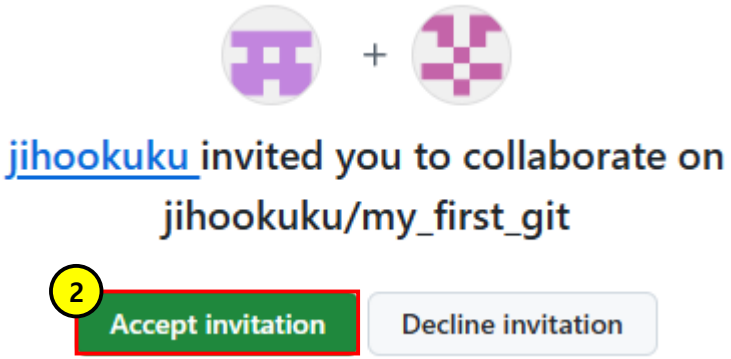
협업

팀원들과 저장소 공유

- Collaborator 에 추가 되면 초대 메일 발송(메일 발송과 관계 없이 초대 가능)
- (1번)Pending Invite 선택 하여 주소 복사
- 해당 주소를 팀원들에게 전달



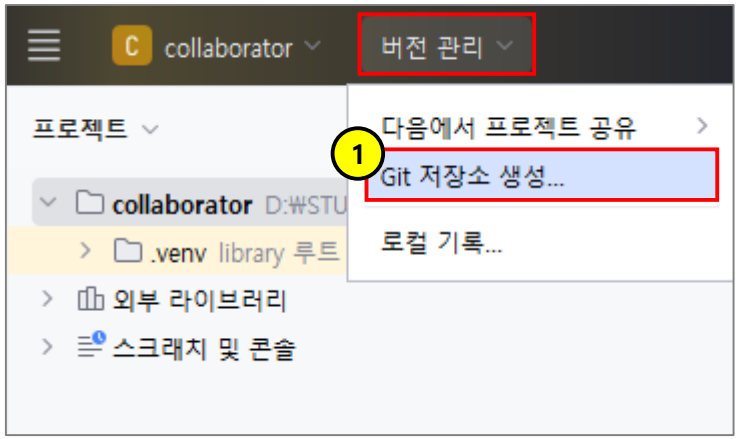
- (2번) invitation accept 선택(이 과정을 하지 않으면 pull 만 가능해짐)

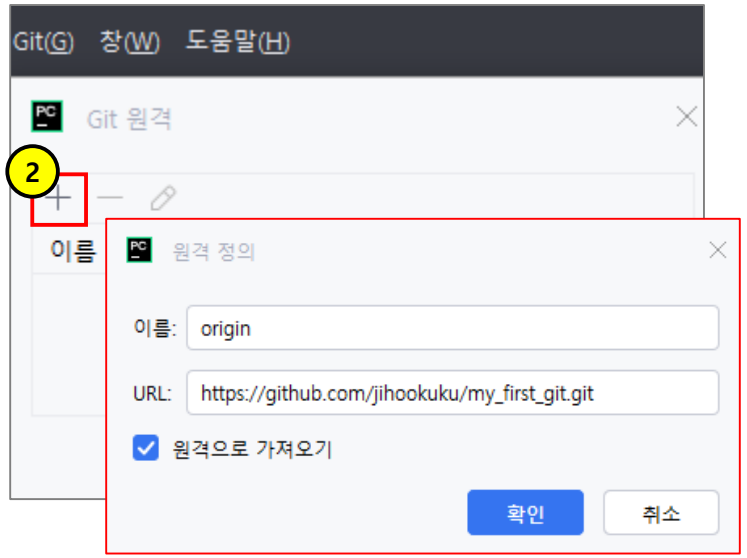


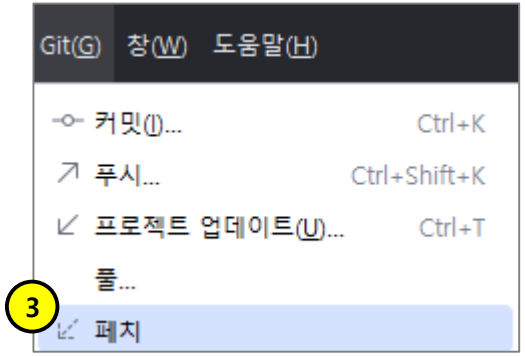
협업

팀원들과 저장소 공유

- (1번)새로운 프로젝트 생성 후 버전관리 > Git 저장소 생성 > 해당 프로젝트 폴더 선택
- (2번) 메뉴 > Git > 원격 관리 에서 공유 받을 주소 입력
- (3번) Git > 페치를 통해 동기화 진행



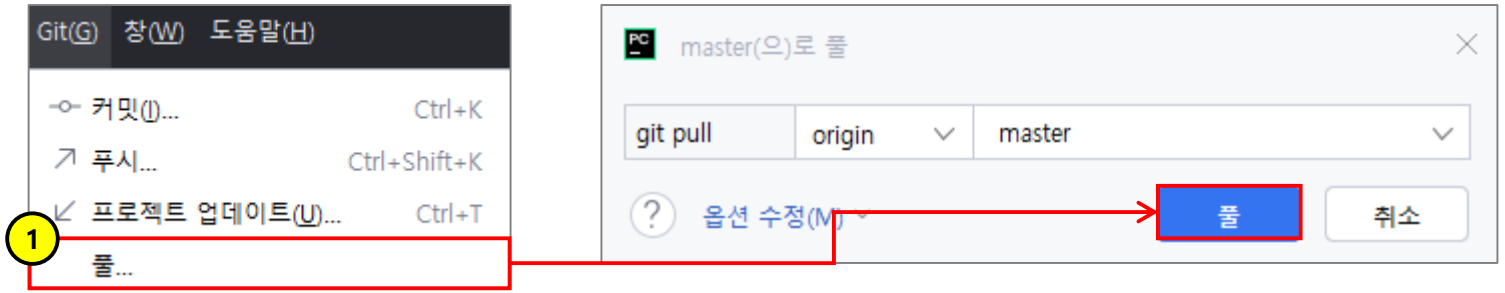




협업

팀원들과 저장소 공유

- (1번) Git > 풀...
- 이때 .idea 폴더 내 .ignore 파일 때문에 에러가 발생하는 경우가 있음(발생시 지워주자)
- (2번) 콜라보레이션 성공 시 나타나는 commit history





Summary

원격저장소

- Git 은 commit 은 로컬에 저장하지만 push 를 통해 원격 저장소에 공유 가능
- 이때 원격 저장소 서비스를 활용 해야함(Git hub, Bit bucket, Get Lab 등)



협업

- 원격 저장소를 팀원들과 함께 작업을 위해서 collaborator 등록이 필요
- 이후 patch 와 pull 을 통해 공유 저장소를 동기화 해야함

